



University
of Exeter

COMM113

Deep Learning – Week 7

Object Detection and Image Segmentation

Gaojie (Jay) Jin
g.jin@exeter.ac.uk

Beyond Image Classification

CNNs

- Previous lecture: image classification

Why is classification alone not enough for real-world tasks?

Beyond Image Classification

CNNs

- Previous lecture: image classification

Why is classification alone not enough for real-world tasks?

Limitations

- Mostly on centred images
- Only a single object per image
- Not enough for many real world vision tasks

E.g., locate objects, not just identify them

Computer Vision Tasks

Classification



CAT

No spatial extent

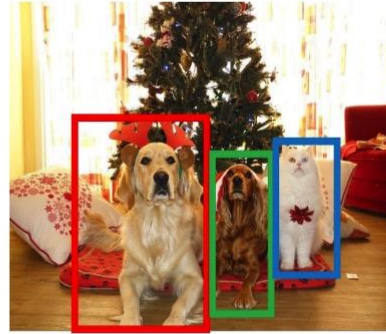
Semantic Segmentation



**GRASS, CAT,
TREE, SKY**

No objects, just pixels

Object Detection



DOG, DOG, CAT

Multiple Object

Instance Segmentation



DOG, DOG, CAT

[This image is CC0 public domain](#)

Which is harder, detection or classification? Why?

Computer Vision Tasks

Classification



CAT

No spatial extent

Semantic Segmentation



**GRASS, CAT,
TREE, SKY**

No objects, just pixels

Object Detection



DOG, DOG, CAT

Multiple Object

Instance Segmentation



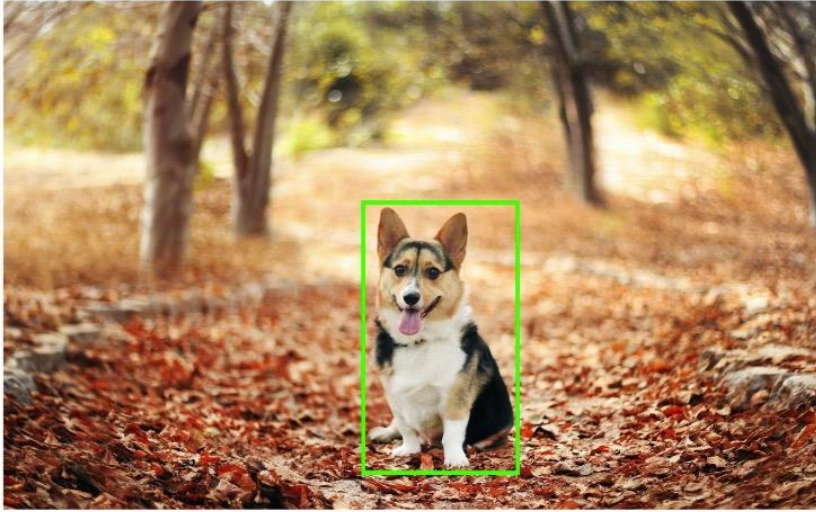
DOG, DOG, CAT

[This image is CC0 public domain](#)

Which is harder, detection or classification? Why?

Detection is harder — must predict bounding boxes + categories.

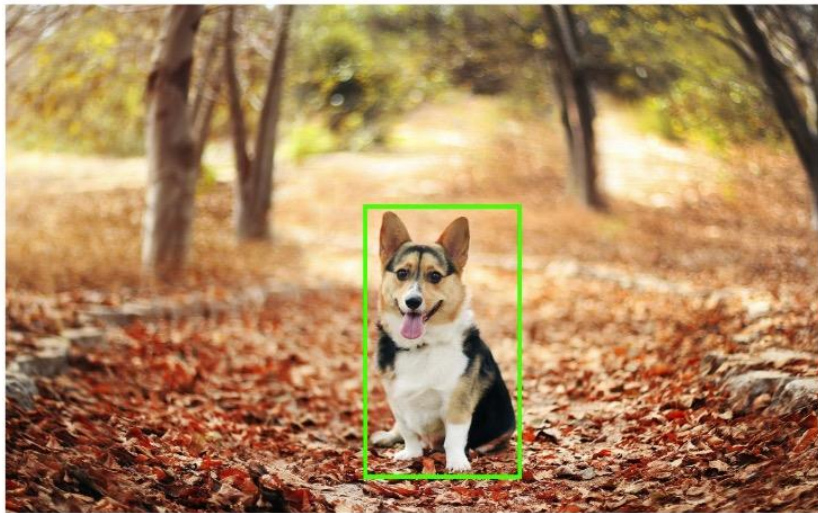
Object Detection - Localisation



Why can't we use accuracy as evaluation for bounding boxes?

- Single object per image
- Predict coordinates of a bounding box (x, y, w, h)
- Evaluate via Intersection over Union (IoU)

Object Detection - Localisation

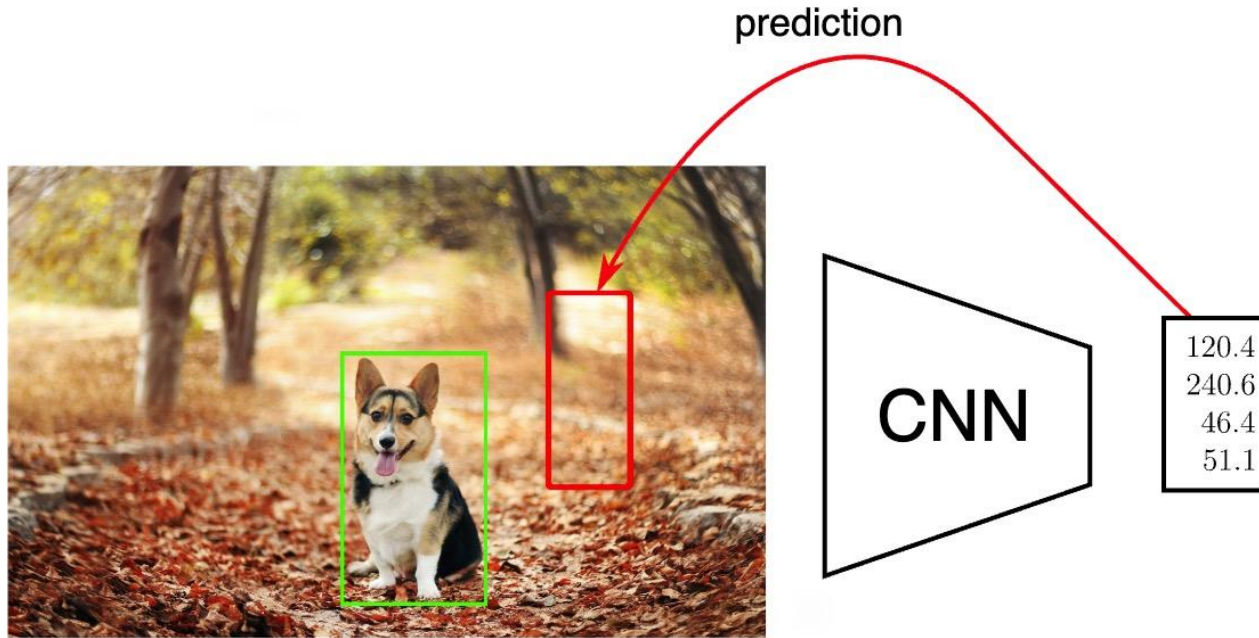


Why can't we use accuracy as evaluation for bounding boxes?

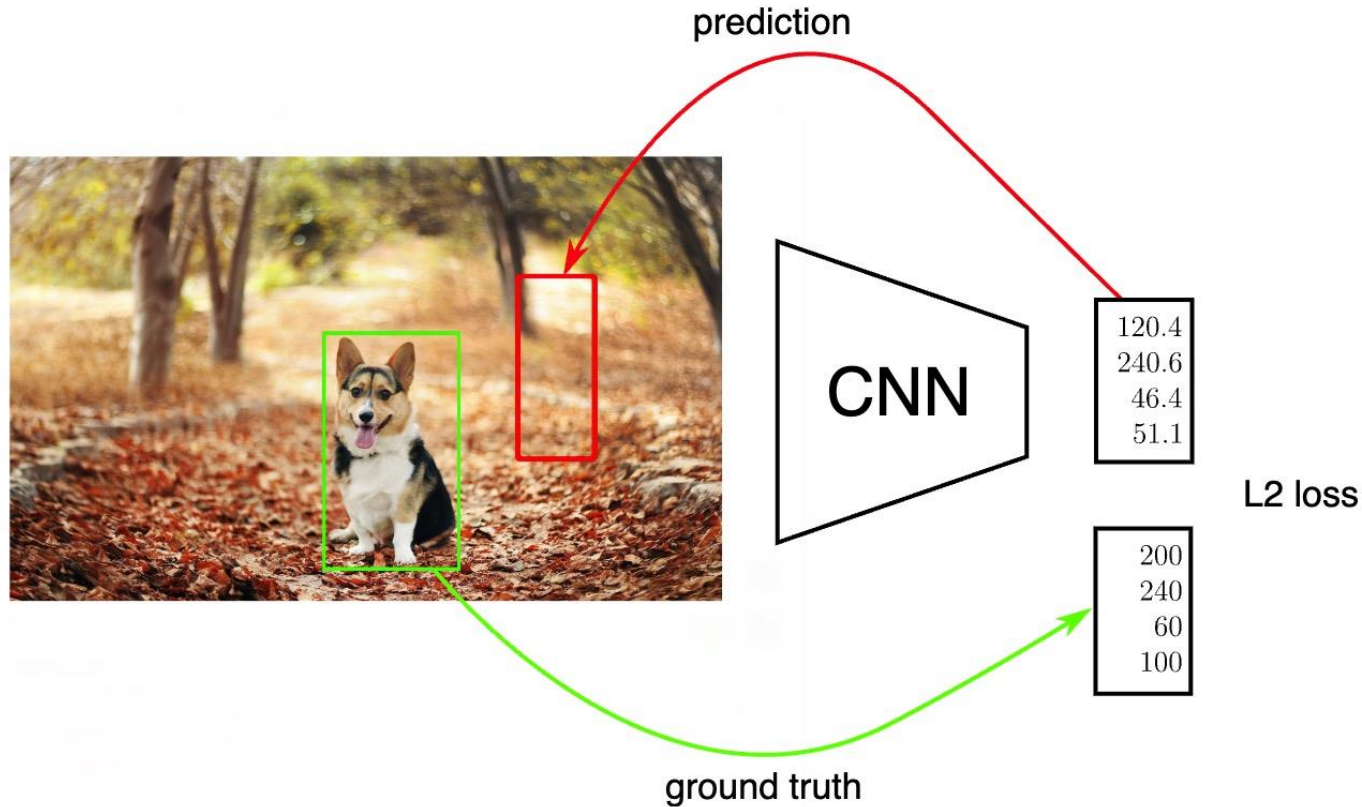
Because bounding boxes are continuous — need overlap measure.

- Single object per image
- Predict coordinates of a bounding box (x , y , w , h)
- Evaluate via Intersection over Union (IoU)

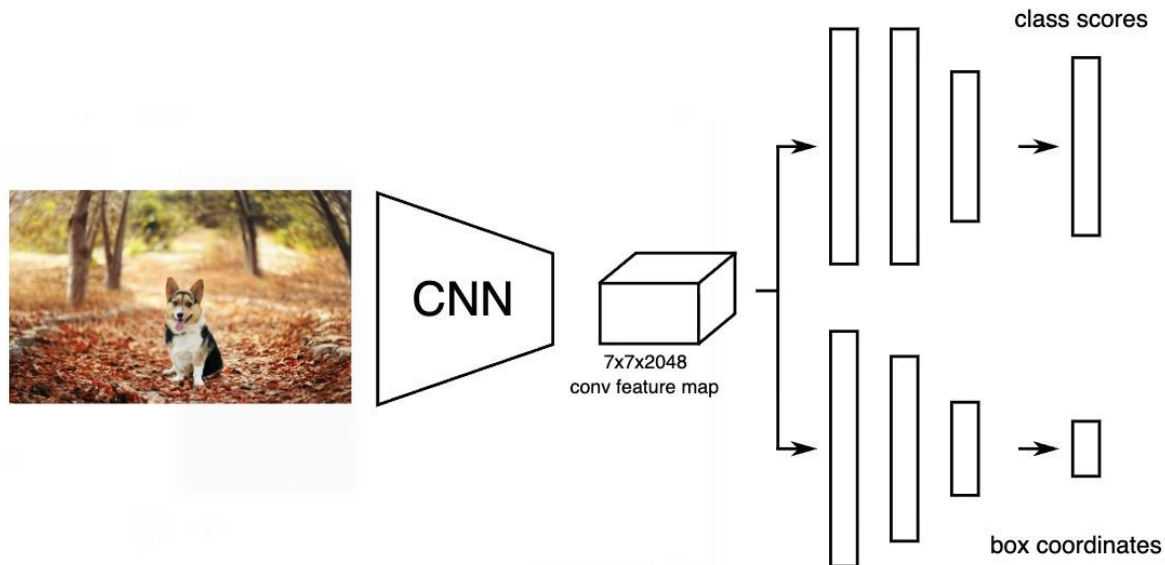
Localisation as regression



Localisation as regression



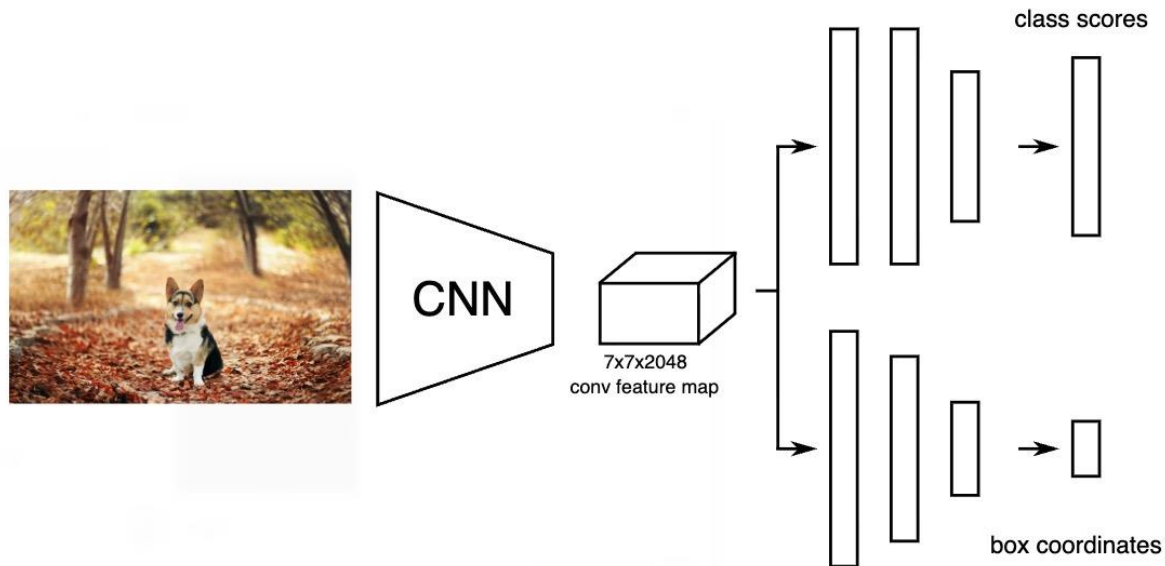
Classification + Localisation



Why is training the regression head harder than the classification head?

- Use a pre-trained CNN on ImageNet (ex. ResNet)
- The "localisation head" is trained separately with regression
- Possible end-to-end finetuning of both tasks
- At test time, use both heads

Classification + Localisation

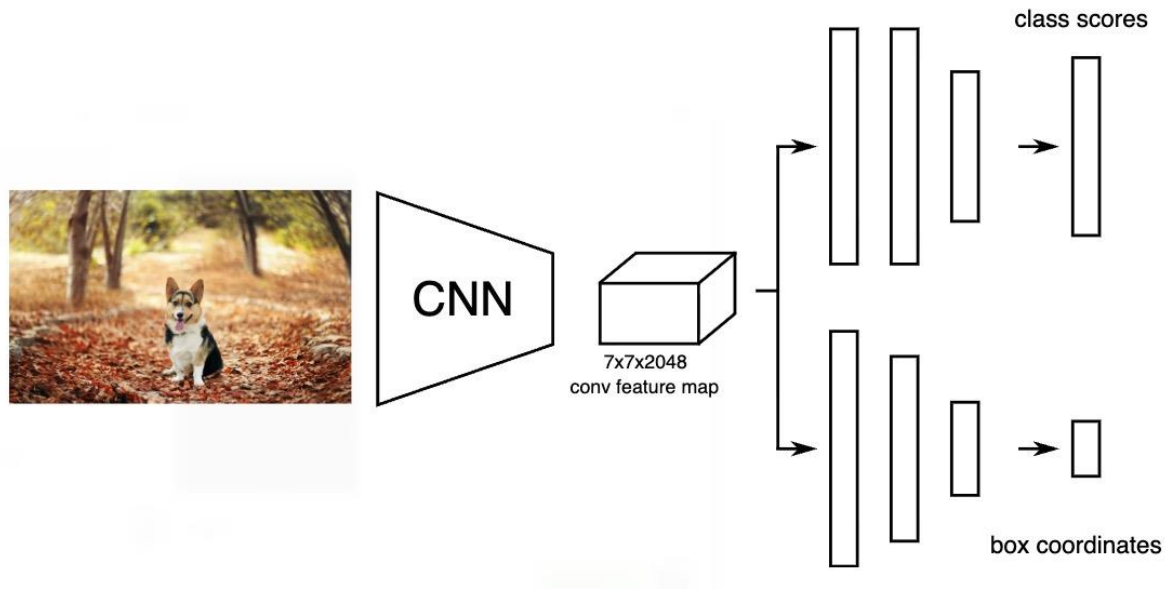


Why is training the regression head harder than the classification head?

- Use a pre-trained CNN on ImageNet (ex. ResNet)
- The "localisation head" is trained separately with regression
- Possible end-to-end finetuning of both tasks
- At test time, use both heads

Regression is sensitive to scale and alignment; classification is more stable.

Classification + Localisation



- K classes, 4 output dimensions (1 box)
- Predict exactly N objects: predict $(N \times 4)$ coordinates and $(N \times K)$ class scores

Object detection

- We don't know in advance the number of objects in the image. Object detection relies on object proposal and object classification
- **Object proposal:** find regions of interest (Rois) in the image
- **Object classification:** classify the object in these regions

Object detection

- We don't know in advance the number of objects in the image. Object detection relies on object proposal and object classification
- **Object proposal:** find regions of interest (Rois) in the image
- **Object classification:** classify the object in these regions

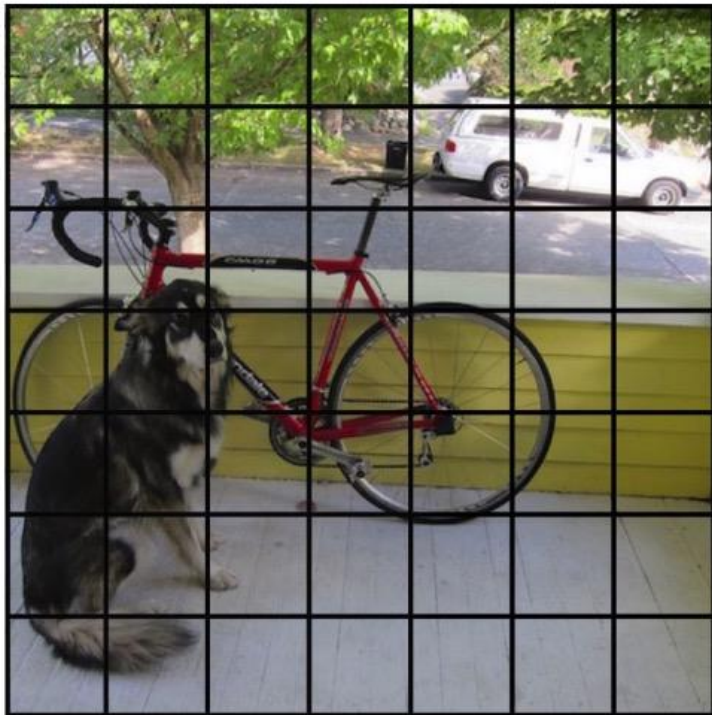
Two main families:

Which do you expect to be faster in real-time?

- Single-Stage: A grid in the image where each cell is a proposal (SSD, YOLO, RetinaNet)
- Two-Stage: Region proposal then classification (Faster-RCNN)

YOLO

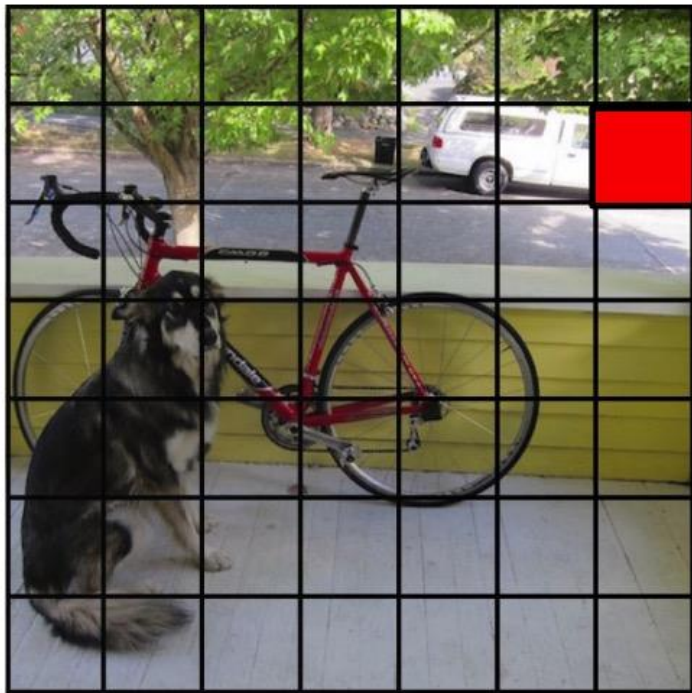
We split the image into an $S \times S$ grid



7*7 grid

YOLO

Each cell predicts B boxes (x,y,w,h) and confidences of each box: $P(\text{Object})$



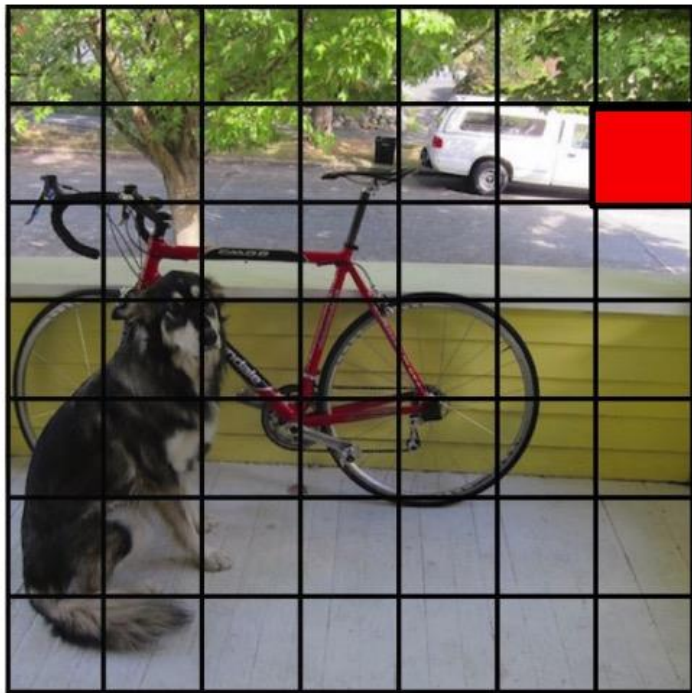
(x,y) : centre of the box, w : width, h : height

(x,y) must locate in the cell

Why force the box centre to lie in the cell, not the entire box?

YOLO

Each cell predicts B boxes (x,y,w,h) and confidences of each box: $P(\text{Object})$



(x,y) : centre of the box, w : width, h : height

(x,y) must locate in the cell

Why force the box centre to lie in the cell, not the entire box?

It prevents multiple cells from claiming the same object.

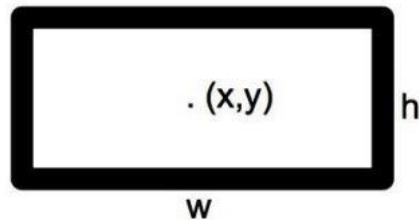
YOLO

Each cell predicts B boxes (x,y,w,h) and confidences of each box: $P(\text{Object})$

$B = 2$



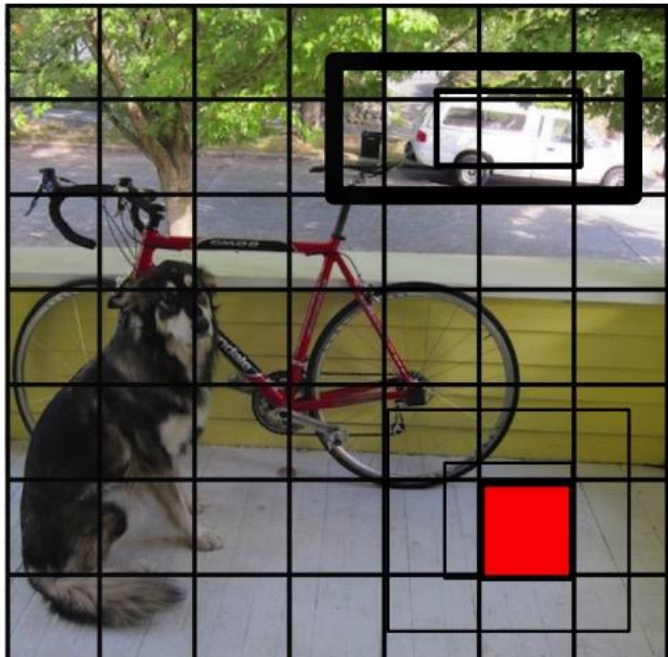
each box predict:



$P(\text{Object})$: probability that
the box contains an object

YOLO

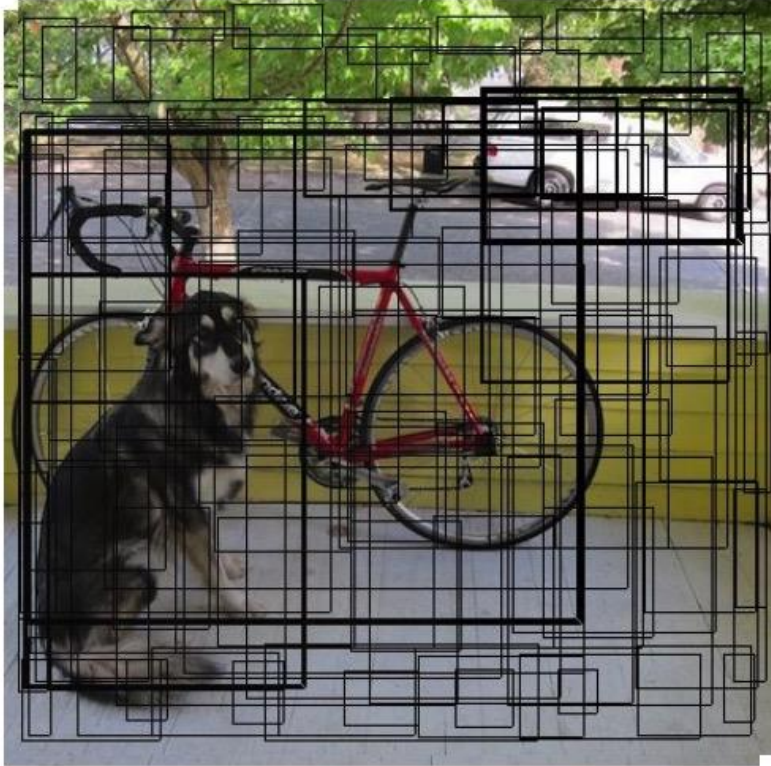
Each cell predicts B boxes(x,y,w,h) and confidences of each box: $P(\text{Object})$



Next cell, ...

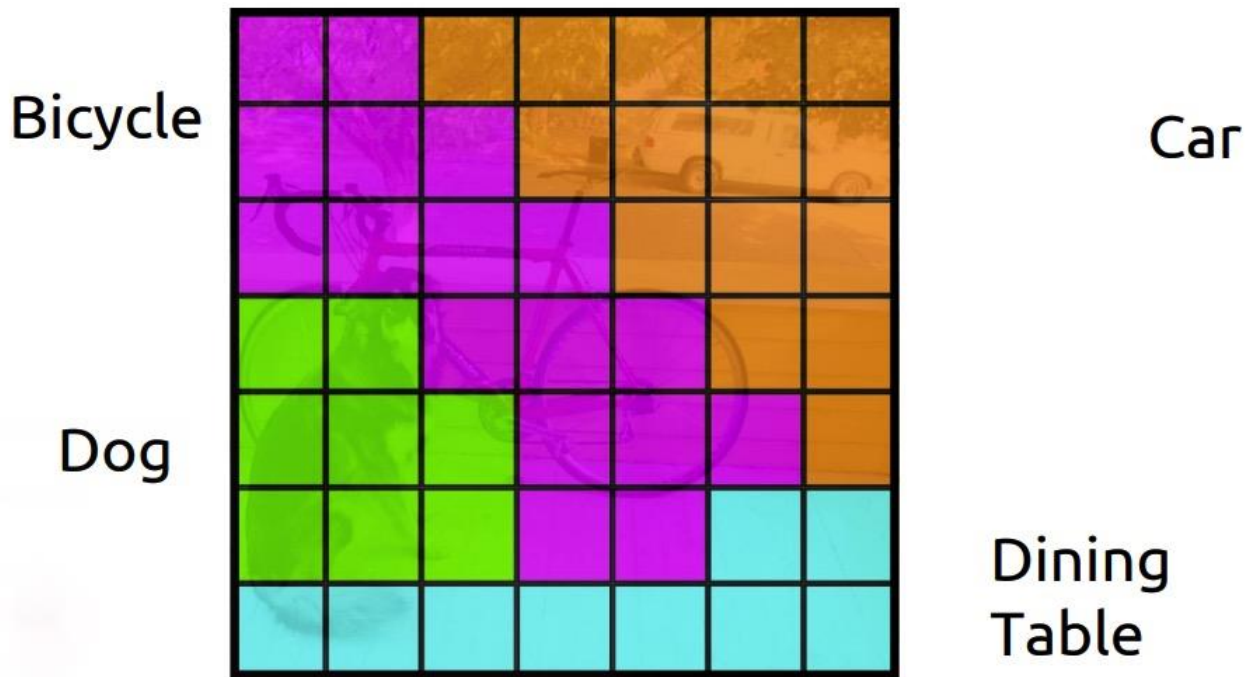
YOLO

Each cell predicts B boxes(x,y,w,h) and confidences of each box: $P(\text{Object})$



YOLO

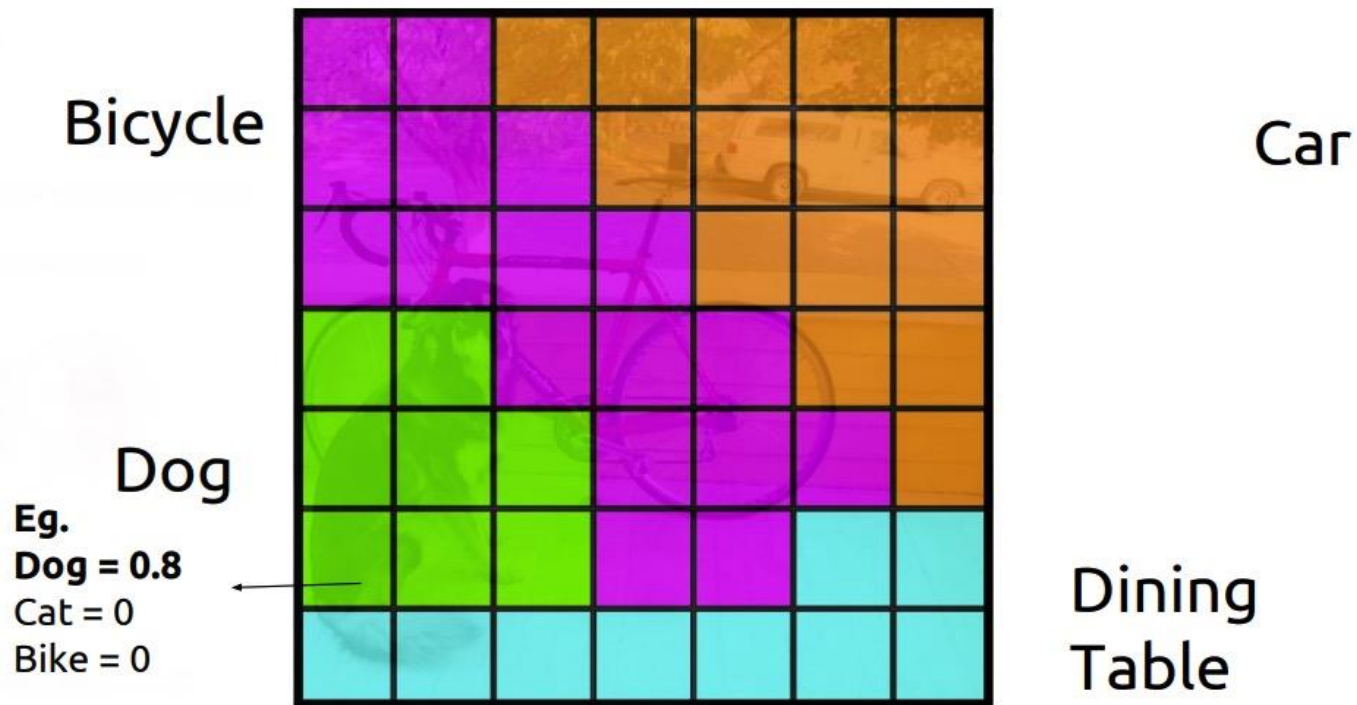
Each cell also predicts a class probability.



Dining Table

YOLO

Conditioned on object: $P(\text{Car} \mid \text{Object})$



YOLO

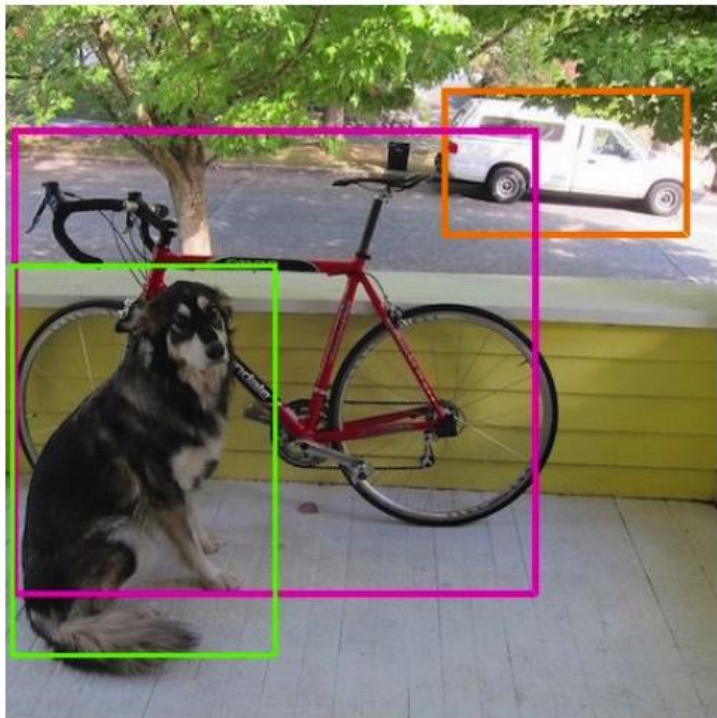
Then we combine the box and class predictions.



$$P(\text{class}|\text{Object}) * P(\text{Object}) \\ = P(\text{class})$$

YOLO

Finally we do threshold detections



Then apply Non-Maximum Suppression (NMS) to remove overlapping duplicates.

YOLO - How to train it?

In YOLOv1, the image is divided into an $S \times S$ grid (typically 7×7).

Each grid cell predicts:

- **B bounding boxes** (typically $B = 2$), each with:
 - x, y : center coordinates (relative to the grid cell)
 - w, h : width and height (relative to the whole image)
 - C : confidence score (IoU between prediction and ground-truth)
- **Class probabilities** (for K classes)

YOLO - How to train it?

(1) Localization Loss for Box Center

- Only apply to boxes **responsible for detecting an object**.
- Measures **how accurate the center position** of the predicted bounding box is.
- Weighted by $\lambda_{coord} = 5$ to emphasize accurate localization.

Purpose: Make sure the box is centered on the object.

YOLO - How to train it?

(1) Localization Loss for Box Center

$$\lambda_{coord} \sum 1_{ij}^{obj} [(x - \hat{x})^2 + (y - \hat{y})^2]$$

- Only apply to boxes **responsible for detecting an object**.
- Measures **how accurate the center position** of the predicted bounding box is.
- Weighted by $\lambda_{coord} = 5$ to emphasize accurate localization.

Purpose: Make sure the box is centered on the object.

YOLO - How to train it?

(2) Localization Loss for Box Width and Height

- Also only for responsible boxes.
- **Square root** is used to:
 - Reduce sensitivity for **large boxes**
 - Increase sensitivity for **small boxes**

Purpose: Make box size accurate while preventing instability.

YOLO - How to train it?

(2) Localization Loss for Box Width and Height

$$\lambda_{coord} \sum 1_{ij}^{obj} [(\sqrt{w} - \sqrt{\hat{w}})^2 + (\sqrt{h} - \sqrt{\hat{h}})^2]$$

- Also only for responsible boxes.
- **Square root** is used to:
 - Reduce sensitivity for **large boxes**
 - Increase sensitivity for **small boxes**

Purpose: Make box size accurate while preventing instability.

YOLO - How to train it?

(3) Confidence Loss (Object Exists)

$$\sum 1_{ij}^{obj} (C - \hat{C})^2$$

- Applies **only** when the cell actually contains an object.

Purpose: Encourage high confidence **only** when the **box** matches the **object**.

YOLO - How to train it?

(4) Confidence Loss (No Object Exists)

$$\lambda_{noobj} \sum 1_{ij}^{noobj} (C - \hat{C})^2$$

- Applies when there is **no object** in the cell/box.
- Weighted by $\lambda_{noobj} = 0.5$, which is **small**.
 - Because most grid cells are background → prevent background overwhelming training.

Purpose: Make model **confidently predict zero confidence** in background areas.

YOLO - How to train it?

(5) Classification Loss

$$\sum 1_i^{obj} \sum_{c=1}^K (p(c) - \hat{p}(c))^2$$

- Only applied **once per grid cell** that contains an object.
- Predicts the **class probabilities** of the object.

Purpose: Teach the model **what** the object is.

YOLO - How to train it?

$$\begin{aligned} L = & \lambda_{coord} \sum_{i=1}^{S^2} \sum_{j=1}^B 1_{ij}^{obj} [(x_{ij} - \hat{x}_{ij})^2 + (y_{ij} - \hat{y}_{ij})^2] \\ & + \lambda_{coord} \sum_{i=1}^{S^2} \sum_{j=1}^B 1_{ij}^{obj} \left[(\sqrt{w_{ij}} - \sqrt{\hat{w}_{ij}})^2 + (\sqrt{h_{ij}} - \sqrt{\hat{h}_{ij}})^2 \right] \\ & + \sum_{i=1}^{S^2} \sum_{j=1}^B 1_{ij}^{obj} (C_{ij} - \hat{C}_{ij})^2 \\ & + \lambda_{noobj} \sum_{i=1}^{S^2} \sum_{j=1}^B 1_{ij}^{noobj} (C_{ij} - \hat{C}_{ij})^2 \\ & + \sum_{i=1}^{S^2} 1_i^{obj} \sum_{c=1}^K (p_i(c) - \hat{p}_i(c))^2 \end{aligned}$$

 1_{ij}^{obj}

The **j-th box in cell i** is responsible for the ground-truth object

 1_{ij}^{noobj}

The cell/box **does not contain** an object

YOLO

What YOLO Does

YOLO

What YOLO Does

- YOLO performs **object detection in a single forward pass**.
- It predicts **what** the objects are and **where** they are simultaneously.

Grid-Based Prediction

YOLO

What YOLO Does

- YOLO performs **object detection in a single forward pass**.
- It predicts **what** the objects are and **where** they are simultaneously.

Grid-Based Prediction

- The input image is divided into an **$S \times S$ grid**.
- **Each grid cell** is responsible for detecting an object **if the object's center falls inside the cell**.

YOLO

Outputs per Grid Cell

Each cell predicts:

YOLO

Outputs per Grid Cell

Each cell predicts:

1. Bounding box coordinates

- (tx, ty) → offset of object center **relative to the grid cell**
- (w, h) → width and height (often **sqrt encoded** for stability)

2. Confidence score:

Probability that an object exists in this cell → $P(\text{Object})$

3. Class probabilities (for C classes):

$P(\text{Class} \mid \text{Object})$

→ Final detection score = $P(\text{Object}) \times P(\text{Class} \mid \text{Object})$

YOLO

YOLO Training Objective

YOLO uses a **multi-part loss**:

- **Localization loss** (for bounding box accuracy)
- **Confidence loss** (object vs. no-object regions)
- **Classification loss** (correct category)

Post-Processing

After prediction:

1. **Filter** low-confidence boxes.
2. Apply **Non-Maximum Suppression (NMS)** to remove duplicates and keep the best bounding box per object.